VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY

Data Mining and Business Intelligence

Market Basket Analysis

Proof of Concept

Association Rules for Retail Recommendations
and Merchandising Decisions

Prototype component	**Implemented evidence**
Input	User-selected Kaggle grocery CSV or ZIP
Mining	Apriori and FP-Growth with equivalence checks
Evaluation	Support confidence lift and held-out stability
Application	Explainable missing-product recommendations
Delivery	Jupyter notebook with generated result files

Problem → Data → Preprocessing → Algorithm
→ Results → Evaluation → Insights → Decision

Executive Summary and Document Guide

This proof of concept demonstrates that a grocery transaction dataset can be converted into evaluated association rules and an explainable basket recommender. The implemented notebook imports user-selected Kaggle data, cleans labels, encodes product presence, runs two frequent-itemset algorithms and exports evidence for review.

The verified baseline uses 9,835 baskets and 169 categories. At support 1% and confidence 25%, Apriori and FP-Growth each identify 333 frequent itemsets. Rule generation produces 170 rules, of which 169 have lift above 1. Every baseline rule's support, confidence and lift is independently checked against transaction counts.

The application recommends yogurt for a basket containing curd and whole milk, with a matching historical rule of approximately 38.52% confidence and 2.76 lift. A separate 80/20 stability experiment finds that 8 of 10 shortlisted rules pass the defined validation screen. This qualification matters: an attractive full-data rule is not automatically deployment-ready.

The prototype proves analytical and application feasibility. It does not demonstrate incremental sales or profit, because no controlled store trial has been performed. The recommended next step is a small merchandising experiment with financial and operational evaluation.

Document guide

Topic	Page
Problem objectives and scope	3
Dataset and input contract	4
Preprocessing and data audit	5
Solution architecture	6
Algorithms and configuration	7
Implementation modules	8
Evaluation measures and worked example	9
Baseline results and interpretation	10
Visual evidence	11
Efficiency and threshold sensitivity	12
Held-out validation	13
Recommendation application	14
Verification and feasibility	15
Reproduction and exported evidence	16
Business decisions limitations and future scope	17
Conclusion and references	18

1 Problem Objectives and Scope

1.1 Problem statement

A grocery retailer records many product purchases but may not systematically identify combinations purchased together. Sales totals explain popularity, while the merchandising question concerns relationships: which combinations have sufficient reach and affinity to justify a recommendation, bundle or coordinated display?

If a manager selects an offer only because both products are popular, the combination may not represent distinctive behaviour. Similarly, a high-affinity pattern involving very few baskets may have limited commercial reach. The prototype therefore combines frequency, conditional reliability and relative association with an interpretable business application.

1.2 Objectives

The implementation must import the specified real dataset, preserve valid transaction boundaries, document preprocessing, and identify frequent itemsets. It must justify and execute Apriori and FP-Growth under equivalent settings, generate rules, verify their metrics, provide visualisations and produce explainable missing-product suggestions.

The business objective is to help a retailer select evidence-based experiments. Increased basket value, relevant offers and improved profit are intended outcomes to measure after a pilot, not results inferred directly from rule strength.

1.3 Included capabilities

The working prototype includes manual CSV/ZIP selection, format checks, Boolean encoding, two mining methods, rule filtering and ranking, repeated timing, threshold sensitivity, a held-out stability check, an editable basket example, an optional interactive picker, edge-case checks and result exports. Every one of its 19 executable cells has a preceding description.

1.4 Scope boundaries

The notebook is a batch academic prototype. It does not connect to a live checkout system, maintain customer accounts, process online orders, train a personalised model or deploy a recommendation API. The basket representation does not establish prices, margins, quantities or customer history. A rule is a co-purchase association rather than a causal or sequential claim.

1.5 Acceptance approach

Success is assessed through correct data dimensions, agreement between algorithm outputs, independently verified metrics, reproducible exports and correct recommendation matching. Held-out screening is assessed separately from these correctness checks. Commercial acceptance requires additional retailer data and a controlled experiment.

2 Dataset and Input Contract

2.1 Dataset source and suitability

The selected download is the Groceries Market Basket Dataset hosted on Kaggle under irfanasrullah/groceries [1]. The underlying canonical Groceries case study is described in arules as 30 days of point-of-sale transactions from a local grocery outlet, containing 9,835 baskets and 169 aggregated categories [2].

Property	**Verified meaning**
Transaction	One shopping basket containing purchased categories
Categories	169 categories including whole milk yogurt and root vegetables
Records	9,835 baskets retained after cleaning
Representation	Variable-length product lists or the equivalent wide CSV
Analytical unit	Basket-level item presence rather than an individual customer

2.2 Supported input formats

The importer always asks the user to select one file. Colab opens its browser upload picker. Local Jupyter opens a file dialog where supported, with a full-path prompt when a graphical dialog is unavailable. The notebook does not search Dataset/ automatically and does not silently replace the selected data.

The headerless groceries.csv contains one product list per row. The alternative groceries - groceries.csv contains an Item(s) count column and product columns; the count, header and blank padding are excluded from mined products. The importer checks the declared count for each wide row. A selected ZIP is read directly, preferring its inner groceries.csv.

2.3 Input validation and provenance

After parsing, the code verifies the expected basket and category counts. A customer/date grocery table with Member_number, Date and itemDescription columns is rejected because it is a different dataset requiring transaction reconstruction. Empty, malformed or incompatible files produce explicit messages rather than an invented successful result.

The notebook records the selected filename, input format, Kaggle source URL and CSV SHA-256 checksum. The tested headerless CSV fingerprint is:

9f6ff7186d88eebe08904798a21d477a2cff475ae145f993e80db80ef567dfeb

Equivalent CSV layouts can have different byte-level fingerprints while representing the same transactions. The checksum identifies the exact CSV bytes; logical equivalence requires comparing their parsed baskets.

3 Preprocessing and Data Audit

3.1 Cleaning procedure

Product tokens are trimmed and lowercased to prevent formatting differences from splitting a category. Empty tokens are removed where present. Within a basket, repeated occurrences of the same product are reduced to one indicator because the analysis models membership rather than quantity. Labels are not merged based on unverified similarity.

Independent transactions with identical contents remain separate baskets. Removing them solely because they match would change product frequencies and support. The audit distinguishes repeated contents from established accidental duplicate records.

Audit field	Observed count
Raw baskets	9,835
Normalised product tokens	408
Empty tokens removed	0
Within-basket repetitions removed	0
Empty baskets excluded	0
Retained baskets	9,835
Distinct cleaned categories	169
Repeated basket contents retained beyond unique combinations	2,824

3.2 Encoding

TransactionEncoder learns the category vocabulary and converts each basket into a Boolean row [5]. True means the category occurs in that transaction and False means it does not. The resulting matrix has 9,835 rows and 169 columns. Numeric scaling is unnecessary because these features are presence indicators.

Illustrative basket	Milk	Bread	Butter
A	True	True	False
B	False	True	True
C	True	False	False

This compact table illustrates the representation; it is not additional measured evidence. The actual matrix uses the full cleaned category vocabulary.

3.3 Data integrity checks

The notebook checks matrix shape, Boolean types, unique columns, absence of missing values and non-empty rows. Selected category counts are calculated independently from the basket lists and compared with matrix totals. Cleaned baskets are exported separately; the original selected file is not overwritten.

4 Solution Architecture

The solution is organised as a single reproducible notebook with logical modules. The user supplies the data and an example basket; the pipeline produces evaluated rules, recommendations and exportable evidence. No external model or API key is needed for the analysis.

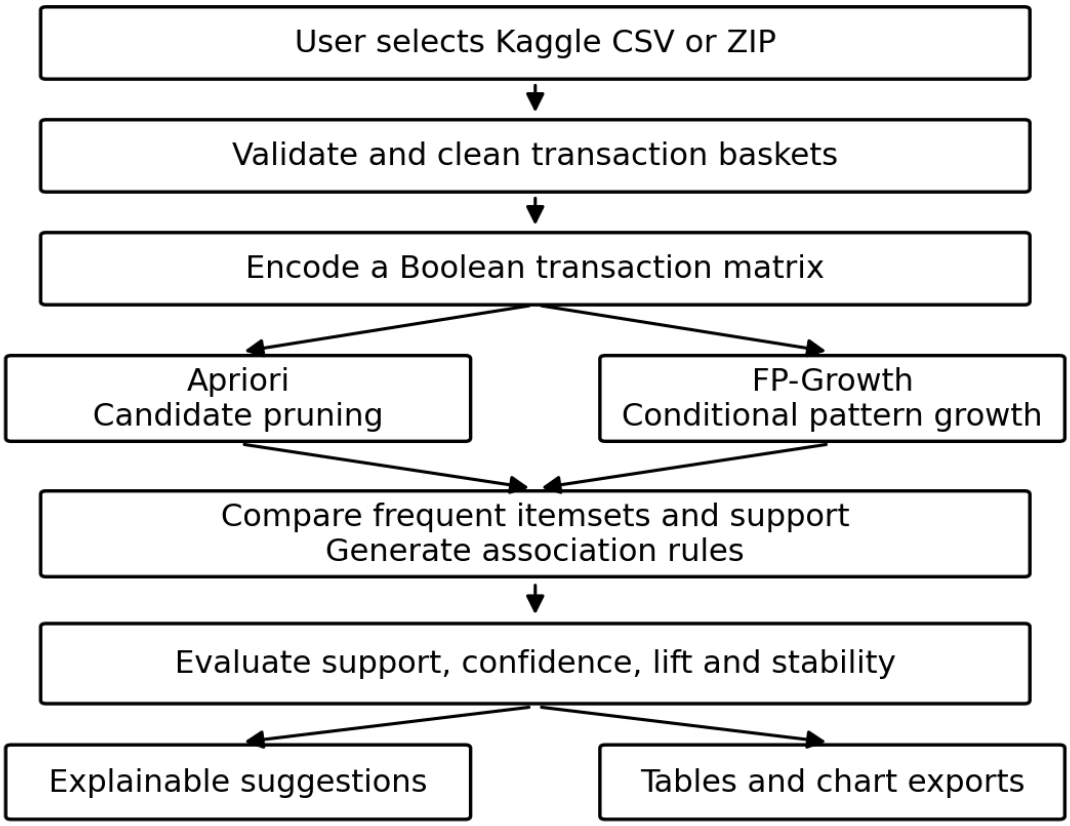


Figure 1 Implemented data processing and association mining architecture

4.1 Module responsibilities

Input and preparation: read selected CSV/ZIP bytes, check the dataset contract, clean product tokens and encode baskets. **Mining and evaluation:** discover patterns, compare both algorithms, generate rules, verify counts and test stability. **Application and reporting:** match a basket to eligible rules, show evidence and export tables and charts.

4.2 Execution and decision boundary

The diagram branches for comparison of two algorithms, but the implementation executes them sequentially for timing. The recommendation result is generated from mined rules in memory. A proposed store intervention is reviewed after this analytical stage; no automated commercial action is executed by the notebook.

5 Algorithms and Configuration

5.1 Appropriate Data Mining task

Association Rule Mining is an unsupervised descriptive task. The aim is to find relationships among products rather than predict a predefined class label. Frequent-itemset discovery identifies unordered product combinations; subsequent rule generation creates disjoint antecedent and consequent sets.

5.2 Apriori

Apriori counts categories and grows candidate itemsets level by level [3]. Its pruning property states that every subset of a frequent itemset must be frequent. If a pair fails support, a triple containing that pair cannot qualify. This reduces unnecessary counting, although large candidate sets and repeated scans can still be expensive.

It is selected as an interpretable baseline: an examiner or analyst can trace a pattern back to basket counts. The notebook calls the library implementation with the encoded matrix and the configured support threshold.

5.3 FP Growth

FP-Growth compresses shared transaction prefixes into an FP-tree and mines conditional patterns without Apriori-style explicit candidate generation [4]. It is a suitable alternative when overlapping baskets permit compression or candidate enumeration becomes costly. Tree structures consume memory and the benefit depends on the data.

Both methods should identify equivalent frequent itemsets and support values on identical data with equivalent restrictions. The notebook verifies this before comparing speed. Faster mining does not make a rule more profitable or more causal.

Setting	Value	Interpretation
Minimum support	0.01	At least 1% of all baskets
Minimum support count	99	Ceiling of 98.35 baskets
Minimum confidence	0.25	At least 25% of antecedent baskets
Positive rule filter	Lift > 1	Association above baseline popularity
Recommendation consequent	One product	Interpretable missing-product suggestion
Random seed	42	Repeatable random validation split

5.4 Why these settings

The baseline aligns with the case study presentation while retaining enough combinations for discussion. One percent support is a coverage floor, not a universal retail rule. Alternative support and confidence settings are evaluated in Section 10 to expose sensitivity rather than imply that one configuration is commercially optimal.

6 Implementation Modules

The final notebook contains 19 executable cells. Each has a preceding description of its purpose, operations, expected output and interpretation. The following map connects the prototype design to its executable evidence.

Step	Implementation operation	Observable output
1	Configure dependencies parameters and working directory	Version table and Results path
2	Request and parse the selected Kaggle CSV or ZIP	Import confirmation and dimensions
3	Preview imported transactions	Five example baskets
4	Clean labels and preserve valid transactions	Audit and before/after record
5	Encode and validate membership	Boolean matrix and count checks
6	Explore frequencies and basket sizes	Descriptive tables and charts
7	Run and compare Apriori and FP-Growth	Itemset agreement and initial times
8	Generate filter and rank rules	Baseline and positive rule counts
9	Interpret the highest-ranked rule	Dynamic metric explanation
10	Verify metrics with direct counting	All baseline rules checked
11	Calculate an eight-basket example	Transparent teaching calculations
12	Visualise associations	Lift scatter and pair-support figures
13	Benchmark and vary thresholds	Timing and sensitivity tables
14	Evaluate held-out stability	Discovery/validation comparisons
15	Define basket recommendation functions	Matching and ranking behaviour
16	Run an editable basket example	Suggestions and supporting rules
17	Offer optional interactive selection	Category picker and button
18	Check recommendation edge cases	PASS confirmation
19	Export evidence and run summary	Results files and ZIP archive

6.1 Library and compatibility handling

The analysis uses pandas, NumPy, matplotlib and mlxtend. The setup installs missing analysis packages and records their versions. Rule generation checks the installed function signature for the optional num_itemsets argument. Optional widgets are separated from the ordinary editable basket example so the core application remains usable without them.

7 Evaluation Measures and Worked Example

Let N be all baskets, count(A) the baskets containing every item in A, and count(A ∪ B) the baskets containing the combined itemset. The primary rule measures are defined below [6].

Measure	Formula	Meaning
Support	count(A ∪ B) / N	Frequency of the complete combination
Confidence	count(A ∪ B) / count(A)	Fraction of A baskets also containing B
Lift	confidence(A → B) / support(B)	Conditional frequency relative to baseline

Lift above 1 indicates positive association, 1 indicates independence, and below 1 indicates negative association. Confidence is directional. A very popular consequent can produce high confidence without distinctive affinity. A high-lift rule may still affect only a small audience.

7.1 Eight illustrative baskets

ID	Products
1	Bread Butter Milk
2	Bread Butter
3	Bread Milk Eggs
4	Bread Butter Milk Jam
5	Milk Eggs
6	Bread Butter Jam
7	Bread Milk
8	Butter Milk Eggs

With support 3/8, Milk and Bread each occur six times, Butter five and Eggs three. Jam occurs twice and is excluded. Bread + Butter + Milk occurs twice, so the triple fails even though its component pairs are frequent.

Rule	Support	Confidence	Lift
Eggs → Milk	37.5%	100%	1.33
Butter → Bread	50%	80%	1.07
Bread → Milk	50%	66.7%	0.89

Bread → Milk exceeds 60% confidence but has lift below 1 because Milk already occurs in 75% of baskets. The example is separate from the Kaggle dataset and illustrates why confidence alone is insufficient. Perfect confidence in only three antecedent baskets also does not establish future certainty.

8 Verified Baseline Results

8.1 Complete result summary

Measure	Observed result
Retained transactions	9,835
Cleaned categories	169
Apriori frequent itemsets	333
FP-Growth frequent itemsets	333
Rules meeting confidence threshold	170
Rules with lift above 1	169
Positive single-consequent recommendation rules	169
Algorithm itemset and support agreement	Verified
Independent rule metric checks	All 170 baseline rules verified

8.2 Selected computed rules

Antecedent → consequent	Count	Support	Confidence	Lift
citrus fruit + other vegetables → root vegetables	102	1.04%	35.92%	3.30
other vegetables + tropical fruit → root vegetables	121	1.23%	34.28%	3.14
beef → root vegetables	171	1.74%	33.14%	3.04
citrus fruit + root vegetables → other vegetables	102	1.04%	58.62%	3.03
root vegetables + tropical fruit → other vegetables	121	1.23%	58.45%	3.02
other vegetables + whole milk → root vegetables	228	2.32%	30.98%	2.84
curd + whole milk → yogurt	99	1.01%	38.52%	2.76

8.3 Interpretation and decision relevance

The leading rule links citrus fruit plus other vegetables to root vegetables. It has 102 supporting baskets, approximately 1.04% support, 35.92% confidence and 3.30 lift. The conditional root-vegetable frequency is therefore about 3.3 times its overall frequency. Support indicates that the complete combination reaches a limited share of transactions.

This finding can justify considering a targeted produce recommendation or display experiment. It cannot establish a 3.3-fold increase in sales. Counts and exact metrics are exported; rounded percentages should not be used to invent alternative supporting counts.

The 333 itemsets include single categories and larger combinations. They are not 333 recommendations. The one rule with lift at or below 1 remains part of the baseline rule count but is excluded from the positive-affinity recommendation pool.

9 Visual Evidence

Figures 2 and 3 are produced from the verified baseline rule table. They communicate different aspects of the result: relative association strength and the trade-off between reach and conditional reliability. The notebook also exports category-frequency, basket-size and selected pair-support visuals.

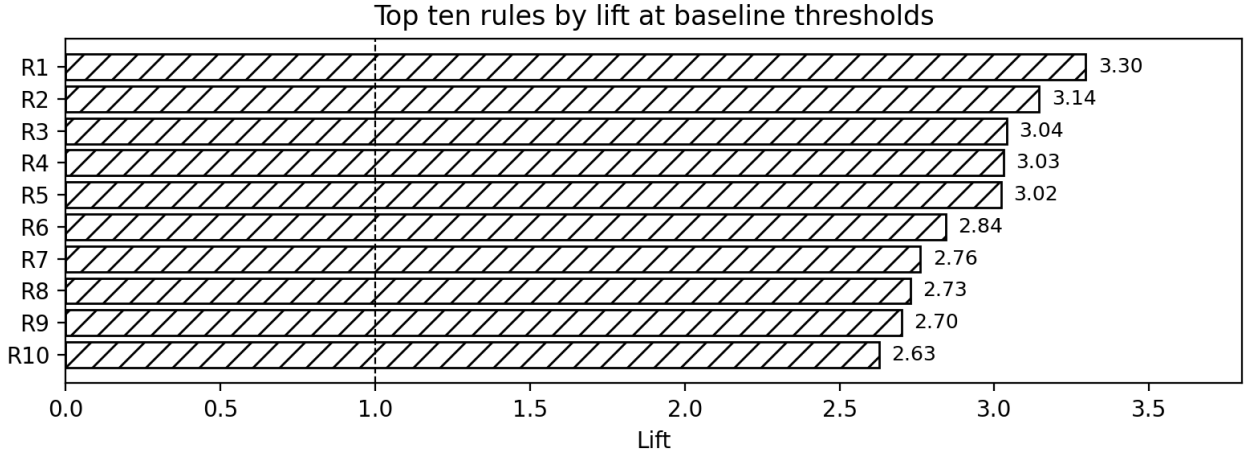


Figure 2 Lift-ranked rule evidence using the baseline analysis

Rule IDs follow descending lift in the exported rule table. R1 is citrus fruit plus other vegetables → root vegetables. The dashed line at lift 1 marks independence. A long bar does not quantify revenue, and the supporting counts must be considered alongside this ranking.

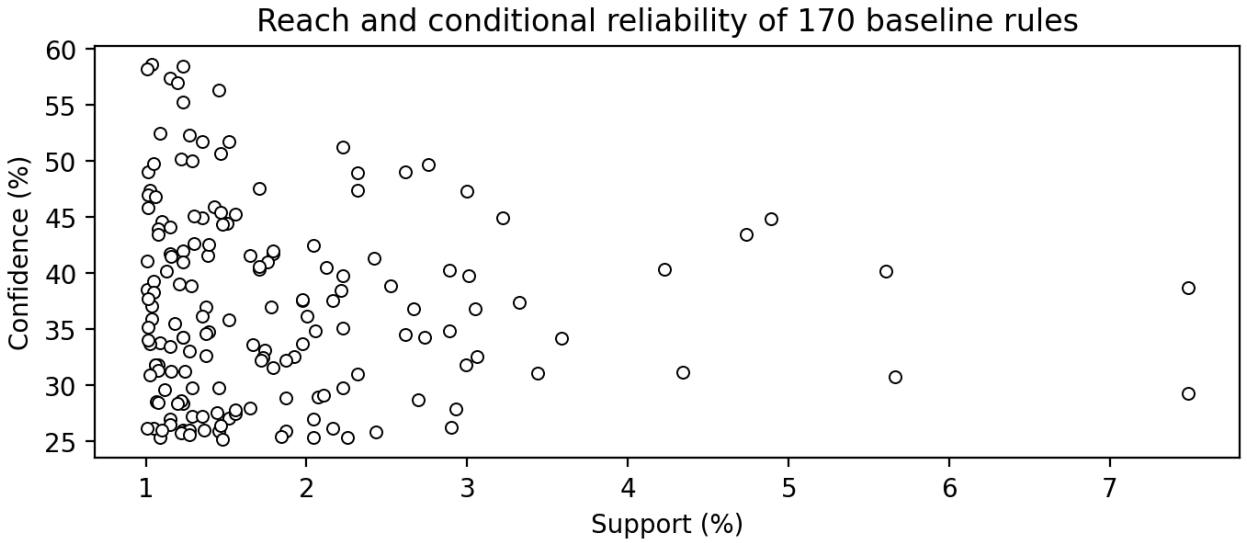


Figure 3 Support versus confidence for all 170 baseline rules

Moving right increases transaction coverage; moving upward increases conditional confidence. Several reliable rules have modest support. A retailer should choose opportunities using sufficient reach, useful affinity, stability and commercial feasibility together rather than select a chart extreme alone.

10 Efficiency and Threshold Sensitivity

10.1 Fair algorithm timing

Both algorithms run on the same cleaned matrix with support 1%. Each is repeated three times and its itemset/support output is checked. The benchmark measures frequent-itemset mining only; preprocessing, rule generation and reporting are outside these times.

Algorithm	Median seconds	Minimum	Maximum	Itemsets
Apriori	0.0466	0.0456	0.0494	333
FP-Growth	0.0819	0.0753	0.1751	333

In this recorded environment, Apriori is faster at the selected threshold. This is a measured project result rather than a contradiction of FP-Growth's design: compression, implementation overhead, data characteristics and thresholds affect performance. No universal algorithm-speed advantage is claimed.

10.2 Sensitivity experiment

Support	Confidence	Itemsets	All rules	Positive rules
0.5%	25%	1001	673	672
0.5%	40%	1001	270	270
0.5%	60%	1001	22	22
1%	25%	333	170	169
1%	40%	333	62	62
1%	60%	333	0	0
2%	25%	122	49	48
2%	40%	122	15	15
2%	60%	122	0	0

Reducing support to 0.5% expands the output to 1,001 itemsets and 673 rules at 25% confidence. Raising support to 2% leaves 122 itemsets and 49 rules at the same confidence. At support 1% and confidence 60%, no rules qualify. An empty output at stricter settings is a valid finding, not a reason to fabricate associations.

10.3 Consequence for decision making

Lower support admits narrower combinations and can increase computational work. Higher confidence filters more rules but can still favour popular consequents. Thresholds should be documented and reviewed against supporting counts and the business objective. The sensitivity table demonstrates this dependence; it does not prove an optimal promotion policy.

11 Held Out Stability Evaluation

11.1 Experimental design

A seeded random split creates 7,868 discovery baskets and 1,967 validation baskets. New rules are mined from discovery baskets only. Ten positive single-consequent rules are selected by discovery lift and confidence, then their metrics are recalculated on validation baskets. This experiment is separate from the descriptive full-data baseline.

The screening condition requires validation lift above 1, confidence at least 25%, and support at least 1%. With 1,967 validation baskets, the minimum integer support count is 20. The flag is an exploratory screen, not a formal significance test or a classification accuracy score.

Rule	Discovery lift	Validation lift	Joint count	Pass
citrus fruit + other vegetables → root vegetables	3.23	3.54	22	Yes
citrus fruit + root vegetables → other vegetables	3.17	2.59	22	Yes
other vegetables + tropical fruit → root vegetables	3.07	3.40	29	Yes
root vegetables + tropical fruit → other vegetables	3.03	2.99	29	Yes
beef → root vegetables	2.99	3.21	42	Yes
whipped/sour cream + whole milk → root vegetables	2.96	1.53	10	No
other vegetables + whole milk → root vegetables	2.94	2.47	40	Yes
other vegetables + yogurt → root vegetables	2.85	2.26	21	Yes
curd + whole milk → yogurt	2.71	2.98	19	No
other vegetables + yogurt → tropical fruit	2.66	2.86	25	Yes

11.2 Interpretation of the two failures

Eight of the ten shortlisted rules pass the screen. Whipped/sour cream plus whole milk → root vegetables has 10 joint validation baskets and approximately 16.95% confidence, failing both the count and confidence conditions. Its validation lift remains above 1, demonstrating why lift alone is insufficient.

Curd plus whole milk → yogurt has 19 supporting validation baskets and approximately 40.43% confidence, with lift about 2.98. It fails the count condition by one basket even though its other metrics remain positive. Its use in the application demonstrates matching logic; it should not be presented as a fully accepted deployment rule under this particular validation policy.

11.3 Generalisation boundary

Random hold-out stability does not demonstrate future-season or another-store performance. A temporal test requires suitable timestamps, which are not established in the selected basket representation. Repeatedly tuning against the same validation subset would weaken its independence.

12 Recommendation Application

The recommender accepts known category names, checks each rule's complete antecedent, excludes purchased products and ranks eligible missing products by lift, confidence and supporting count. It displays the strongest matching rule per product and returns at most five suggestions by default.

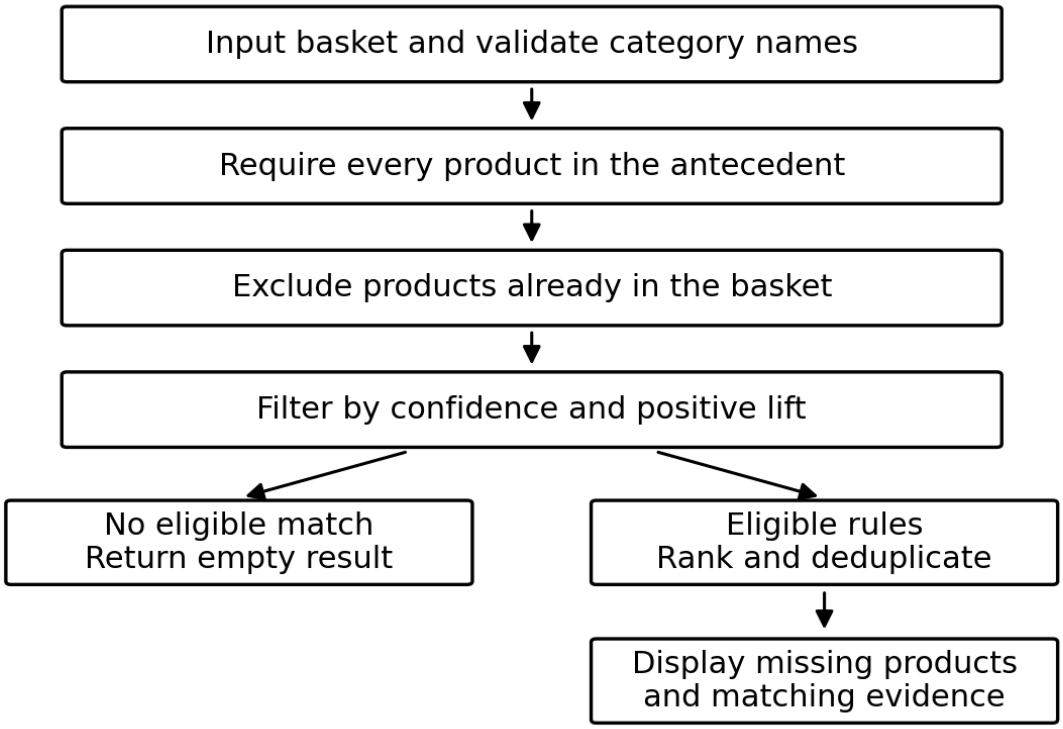


Figure 4 Rule matching and explanation flow in the implemented application

12.1 Executed example

DEMO_BASKET = ["curd", "whole milk"]

Suggested product	Matched antecedent	Count	Confidence	Lift
Yogurt	Curd + whole milk	99	38.52%	2.76
Other vegetables	Curd	169	32.25%	1.67

Curd alone does not satisfy the two-item yogurt rule; both antecedent items must be present. A shorter rule can independently match a subset of the basket, as shown by the other-vegetables suggestion. If yogurt is already present, it is excluded from the recommendation list.

12.2 Interface and interpretation

The editable Python basket is the core interface. An optional multi-select widget calls the same function. Unknown or empty inputs produce clear messages, while a valid no-match basket returns an empty result. Historical confidence is not a calibrated probability for an individual customer, and suggestions are not automatically executed as offers.

13 Verification and Feasibility

13.1 Verification evidence

Check	Observed evidence	Status
Dataset contract	9,835 baskets and 169 categories	Passed
Matrix integrity	Boolean values valid dimensions and counts	Passed
Algorithm agreement	333 matching itemsets and support values	Passed
Rule metrics	All 170 baseline rules independently recalculated	Passed
Antecedent matching	Partial input does not satisfy a multi-item rule	Passed
Purchased-item exclusion	Already-present consequents are not suggested	Passed
Invalid input	Empty and unknown categories are rejected	Passed
Valid no-match input	Empty recommendation result permitted	Passed
Input representations	CSV wide CSV and ZIP parsing checked	Passed
Manual selection	Upload requested even when local data exists	Passed
Result export	17 evidence files and external ZIP verified	Passed
Held-out screening	8 of 10 shortlisted rules pass	Qualified
Commercial uplift	No controlled retailer trial performed	Not evaluated

13.2 Technical feasibility

The tested run uses Python 3.12.14, NumPy 2.3.5, pandas 3.1.0, matplotlib 3.10.8 and mlxtend 0.25.0. The data is manageable as a local Boolean matrix. No paid model API, database service or specialised GPU is required. Runtime and memory needs for a substantially larger retailer dataset remain to be measured.

13.3 Operational feasibility

The output contains readable products, counts and metrics, allowing a manager to inspect why an action is proposed. Retail feasibility still depends on margins, stock, shelf space and trial operations. Adding these inputs is a practical next stage rather than a completed feature.

13.4 Scope of testing

All 19 analysis cells were executed sequentially with the selected Kaggle data, with no error or warning outputs retained. Input-selection paths were tested using controlled upload/dialog responses. This does not certify every browser, operating system or notebook frontend. The specific legacy Jupyter UTC helper repair was separately tested for timezone-aware output without deprecated calls.

14 Reproduction and Exported Evidence

14.1 Reproduction procedure

Place Market_Basket_Analysis_CA2_Final.ipynb in the CA1_CA2_Market_Basket_Analysis project folder. Retain the downloaded Kaggle data under Dataset/ for submission organisation. Open the notebook and restart the kernel or runtime, then run all cells in order.

The import step always requests one CSV or ZIP rather than searching folders. Colab uses Choose Files. Local Jupyter uses a graphical dialog where available, otherwise a full-path prompt. A Windows C:\Users path is not readable in a hosted Colab runtime; upload the file there. Wait for IMPORT SUCCESS and the final IMPLEMENTATION COMPLETED summary.

Run from the intended project directory because Results/ is based on the kernel's working directory. The initial installation of missing packages needs internet. After data and packages are available, the analytical pipeline does not require an external service.

14.2 Export inventory

Evidence group	Files in Results
Charts	01_data_overview.png; 02_top_rules_lift.png; 03_support_confidence.png; 04_pair_support_matrix.png
Mining tables	all_association_rules.csv; positive_rules.csv; frequent_itemsets.csv; item_frequencies.csv
Evaluation tables	algorithm_benchmark.csv; threshold_sensitivity.csv; holdout_validation.csv
Application and cleaned input	demo_recommendations.csv; cleaned_baskets.csv
Audit and provenance	cleaning_audit.json; environment_versions.json; run_summary.json; dataset_source.txt

Market_Basket_Results.zip is saved beside Results/, not inside it, so it does not include itself. Reruns regenerate the standard filenames. The original input is preserved. The POC, PPT, notebook and results should report consistent actual values.

14.3 Environment troubleshooting

The setup suppresses warning messages for readability while leaving genuine failures visible. For older Jupyter clients emitting repeated datetime.utcnow deprecation messages during upload, a guarded session-only helper replacement returns timezone-aware UTC. It does not edit installed package files or upgrade the kernel stack. Restarting and running the current setup is required to apply it in a new session.

If a later cell reports an undefined variable, execute earlier cells in order. If widgets are unavailable, use the ordinary editable basket. If the file dimensions differ, select the specified Kaggle dataset rather than changing counts to make an unrelated file pass.

15 Business Decision and Limitations

15.1 Mapping patterns to actions

Finding	Candidate action	Business evidence needed
Produce rules show positive affinity	Test coordinated display or a relevant produce suggestion	Basket value and profit compared with control
Curd and milk match yogurt rules	Test an eligible-basket recommendation or small offer	Uptake and incremental profit after costs
Popularity can inflate confidence	Use support lift stability and feasibility together	Reject narrow unstable or unprofitable actions

15.2 Proposed controlled pilot

Choose one feasible intervention and define eligible transactions before launch. Use an appropriate comparison group; layout changes may require store-level assignment rather than individual transaction assignment. Establish a primary outcome, trial duration and analysis method before inspecting outcomes. Monitor stockouts and keep other major promotions comparable where practical.

Average basket value needs transaction revenue. Unit-based items per basket needs quantities, whereas this prototype directly measures distinct categories. Incremental profit needs prices, costs and discount expenses. Check whether extra purchases replace other profitable products or merely discount combinations customers would have bought anyway.

15.3 Limitations

The data covers a short period at one outlet, limiting conclusions across stores and seasons. Aggregated categories hide brand and pack-size effects. The selected representation does not establish a customer identifier, transaction timestamps, quantities, prices or margins for the current analysis. Existing merchandising and availability can influence co-purchases.

Rule discovery depends on thresholds and sample composition. A large search can expose chance relationships, and the held-out results already show that some attractive rules fail the screening policy. Association is not causation. Neither lift nor confidence establishes the profit effect of implementing a recommendation.

15.4 Future scope

Add product margin and stock data to prioritise feasible opportunities. Validate across stores and newer periods. Investigate high-utility itemset mining when quantities and profit are available. Sequential analysis requires reliable timestamps and linkage across transactions. Customer-specific targeting would require appropriate customer data and governance; it is not inferred from aggregate baskets here.

Conclusion and References

Conclusion

The proof of concept meets its analytical objective: real grocery baskets are imported, cleaned and encoded; two suitable frequent-itemset algorithms are implemented and compared; association rules are generated and independently evaluated; and a transparent basket recommender demonstrates a practical use of the findings.

The verified baseline contains 333 frequent itemsets and 170 rules, with 169 positive associations. The prototype also exposes uncertainty: only 8 of 10 shortlisted rules pass the defined hold-out screen, and the demonstrated yogurt rule narrowly misses its validation count floor. These results support a carefully reviewed pilot rather than an unconditional store rollout.

The next decision is to select one operationally feasible action and measure it against a control using appropriate financial data. The POC therefore connects the required sequence from problem and data through algorithms, results, evaluation and insight to a testable business decision.

References

[1] Kaggle. Groceries Market Basket Dataset. https://www.kaggle.com/datasets/irfanasrullah/groceries

[2] Hahsler M Hornik K and Reutterer T. Groceries The Groceries Transactions Data Set. arules documentation. https://michael.hahsler.net/arules/reference/Groceries.html

[3] Agrawal R and Srikant R. 1994. Fast Algorithms for Mining Association Rules. VLDB pp 487–499. https://www.vldb.org/conf/1994/P487.PDF

[4] Han J Pei J and Yin Y. 2000. Mining Frequent Patterns without Candidate Generation. ACM SIGMOD. https://sigmodrecord.org/2000/06/07/mining-frequent-patterns-without-candidate-generation/

[5] Raschka S and mlxtend contributors. TransactionEncoder documentation. https://rasbt.github.io/mlxtend/user_guide/preprocessing/TransactionEncoder/

[6] Raschka S and mlxtend contributors. Association rules documentation. https://rasbt.github.io/mlxtend/user_guide/frequent_patterns/association_rules/

[7] Raschka S and mlxtend contributors. Apriori and FP-Growth documentation. https://rasbt.github.io/mlxtend/user_guide/frequent_patterns/apriori/ and https://rasbt.github.io/mlxtend/user_guide/frequent_patterns/fpgrowth/

[8] Project implementation evidence. Market_Basket_Analysis_CA2_Final.ipynb and generated Results tables including run_summary.json algorithm_benchmark.csv threshold_sensitivity.csv and holdout_validation.csv. Verified project run on 31 October 2026.

The computed results and figures in this POC use the verified project outputs. The eight-basket example and controlled test rule are explicitly illustrative. No named retailer deployment or commercial trial result is claimed.